Hydra-Reviewer: A Holistic Multi-Agent System for Automatic Code Review Comment Generation

Xiaoxue Ren, *Member, IEEE*, Chaoqun Dai, Qiao Huang, Ye Wang, Chao Liu, and Bo Jiang

***Abstract*—Review comment generation is a crucial task in code review, and significant progress has been made in automating. Previous research has generated review comments by fine-tuning pre-trained models or Large Language Models (LLMs). However, these studies have overlooked the necessity of conducting code reviews from multiple perspectives, resulting in the omission of potential issues in code changes. Additionally, the complexity of review comments often hinders the accurate quantitative evaluation of automated tools' effectiveness. In this paper, we first conduct an empirical study to propose a comprehensive taxonomy of code review dimensions. We also identify three major limitations of existing automated code review (ACR) methods: lack of comprehensiveness, incorrectness, and vagueness. Building on the insights from our empirical study, we introduce HYDRA-REVIEWER, a collaborative multi-agent framework powered by large language models, designed to automatically generate high-quality code reviews. We utilize the CodeReview and CodeReviewNew benchmark datasets, along with a newly constructed review comment generation dataset. We compare HYDRA-REVIEWER with several baselines, including CodeReviewer, LLaMA-Reviewer, ChatGPT, Comprehensive-ChatGPT, and DeepSeek-V3. The experimental results show that HYDRA-REVIEWER achieves a BLEU score of 8.20, outperforming the state-of-the-art baseline, DeepSeek-V3, which scores 7.85. In qualitative evaluation, HYDRA-REVIEWER's generated comments span an average of 7.8 review dimensions, addressing the limitations of existing ACR methods effectively. Additionally, HYDRA-REVIEWER demonstrates strong generalization capabilities on unseen dataset. We further validate the contributions of each component of HYDRA-REVIEWER through an ablation study and confirm the helpfulness and readability of the generated comments via a User Study. Finally, a cost analysis reveals that HYDRA-REVIEWER generates review comments at an average cost of 0.018 dollars and 62.63 seconds per code change.**

***Index Terms*—Software quality, code review, large language model, multi-agent.**

I. INTRODUCTION

CODE review is a collaborative process in which team members, other than the code author, manually evaluate the code to identify defects and enhance its quality, while also fostering knowledge sharing among the team [1]. It is widely regarded as a fundamental practice within the software development lifecycle [2]. During the code review process, reviewers provide feedback through review comments, which the code author then uses to refine and improve the code. The primary objective of code review is to enhance the overall quality of the software, and its effectiveness in bolstering the reliability and maintainability of software systems is well-documented and broadly accepted [3].

However, code review is a time-consuming process that requires a large amount of manual inspection of the code, especially in projects with large contributions. Bosu et al. found that developers needed to spend an average of 6 hours per week on code review [4]. To reduce developers' workload, many studies focus on automating code review, such as comment location prediction [5], [6] and review comment generation [7]. Among them, learning-based methods [8], [9] that rely on large language models (LLMs) such as CodeT5 [10] and LLaMA [11] demonstrate good results in reducing the manual labor required for code review.

After observation, existing automated code review (ACR) methods face two critical challenges in aligning with real-world software engineering practices. First, they exhibit **a lack of multi-dimensional analysis (C1)**, generating only isolated comments per code change. This contrasts starkly with human review practices, where evaluators systematically address interdependent quality dimensions, such as functionality, security, readability, and maintainability [8]. Second, these methods may generate comments of **inadequate quality (C2)**. Developers expect comments to be relevant to code changes and provide adequate information, but a significant portion of open-source comments deviate from this standard [12]. Although existing ACR methods can generate comments with high BLEU scores by mimicking real-world comments, their training data inherently contains such low-quality examples. Consequently, the

Received 23 March 2025; revised 10 August 2025; accepted 28 September 2025. Date of publication 14 October 2025; date of current version 12 December 2025. This work was supported in part by the Natural Science Foundation of China under Grant 62302437, Grant 62302447, and Grant 62202074; in part by the Fundamental Research Funds for the Central Universities under Grant 226-2025-00004; in part by Zhejiang Province's "Sharpshooter & Pioneer" Key Research and Breakthrough Program under Grant 2024C01070; in part by Tongxiang Institute for General Artificial Intelligence Research Project under Grant TAGI2-B-2024-0015; and in part by the Major Cooperation Projects under the Eagle Plan Cultivation of Zhejiang Provincial Administration for Market Regulation under Grant CY2023109. Recommended for acceptance by T. Zhang. *(Corresponding author: Qiao Huang.)*

Xiaoxue Ren is with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Zhejiang University, Hangzhou 310058, China (e-mail: xxren@zju.edu.cn).

Chaoqun Dai, Qiao Huang, Ye Wang, and Bo Jiang are with the School of Computer Science and Technology, Zhejiang Gongshang University, Hangzhou 310018, China (e-mail: daichaoqun01@gmail.com; qiaohuang@zjgsu.edu.cn; yewang@zjgsu.edu.cn; nancybjiang@zjgsu.edu.cn).

Chao Liu is with the School of Big Data and Software Engineering, Chongqing University, Chongqing 400044, China (e-mail: liu.chao@cqu.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3621462

generated results may replicate the same issues, leading to degraded comment quality and ultimately failing to meet developer expectations. Notably, prior research has yet to systematically define the dimensions prioritized in manual code reviews or evaluate how well ACR-generated comments align with these dimensions. Furthermore, the quality of ACR-generated comments remains uninvestigated.

To address these gaps, we conduct an empirical study to quantify the severity of insufficient multi-dimensional evaluation (related to C1) and comment quality deficiencies (related to C2). Specifically, we first collect 29,612 code changes and their corresponding review comments from the top 1,000 GitHub repositories ranked by star count. Using minimum sampling strategy [13], we randomly select 384 review comments for analysis and develop a hierarchical taxonomy encompassing 19 distinct code review dimensions, such as code logic readability, comment quality, and security compliance. The distribution of comments across these dimensions is quantified to establish a baseline for multi-dimensional evaluation.

Subsequently, we evaluate three state-of-the-art ACR methods (i.e., CodeReviewer, LLaMA-Reviewer, and ChatGPT) by generating comments for the sampled code changes. Through manual comparison of human comments (authored by developers) and ACR-generated comments, we identified critical shortcomings. First, both ACR-generated comments and human comments exhibit insufficient comprehensiveness. Although both can identify code quality issues in some dimensions, they may overlook issues in others. The absence of multi-dimensional analysis can ultimately lead to the merging of code with potential quality issues into the codebase. For example, in Fig. 1(a), a human reviewer critiques variable naming in a loop but overlooks a missing space after "if", violating Python's coding conventions and impairing readability. Second, ACR-generated comments suffer from high error rates: 33.2% of suggestions from CodeReviewer and LLaMA-Reviewer are factually incorrect. Fig. 1(b) illustrates an instance where an ACR method misinterpreted a code comment and proposed an irrelevant modification. Third, vagueness plagues 61.0% of comments from conventional ACR methods compared to 16.4% for ChatGPT. These comments lack the location of the comment, the justification for suggested changes, or actionable feedback, and thus remain misaligned with developer expectations. For instance, the unsubstantiated suggestion "Revert this change" lacks justification, reducing its persuasiveness.

After that, to bridge the gap between automated systems and human-driven reviews, we propose HYDRA-REVIEWER (a **H**olistic multi-agent s**Y**stem for automatic co**D**e **R**eview comment gener**A**tion), a novel framework leveraging GPT-4o-mini. HYDRA-REVIEWER addresses **C1** (multi-dimensionality) and **C2** (comment quality) through three steps: context retrieval, initial comment generation, and comment processing. Context retrieval extracts code snippets related to code changes from source code to provide agents with more background information. Initial comment generation uses multiple agents to review the code changes from different dimensions and generate multi-dimensional comments. Comment processing summarizes the agent-generated comments, filters out low-quality (including incorrect and vague) suggestions, and outputs the final review comments after reordering the suggestions based on priority.

```
+  def make_plugin(self, plugin):
+      plugin_names = []
+      for plugin in plugin:
+          if(plugin is not None):
+              plugin_names.append(plugin.get('name'))
+      return plugin_names
```

Human Reviewer: Avoid using the same variable name in the for loop.

(a)

```
-     logs.log_warn(qemu_log)
+     # Only report the tail of the log; otherwise we would only end up seeing
+     # the beginning of it once the logging library later truncates it to the
+     # STACKDRIVER_LOG_MESSAGE_LIMIT.
+     logs.log_warn(qemu_log[-64 * 1024:])
```

ACR Method: Shouldn't this be 64 * 1024 * 1024?

(b)

Fig. 1. Examples of review comment quality – (a) not comprehensive review comment & (b) incorrect review comment.

We evaluate HYDRA-REVIEWER on both CodeReview [8] and CodeReview-New [3] datasets, which contain a total of 28,049 code changes. We crawl all the human comments corresponding to the code changes. Since some comments might be irrelevant to code quality, which affects the validity of the evaluation, we filter the comments and ultimately retain 18,138 code changes and their corresponding 25,323 human comments. Experimental results show that our HYDRA-REVIEWER outperforms the baselines by an average of 31.3% in BLEU-4 score and 15.3% in the number of dimensions. We also collect data beyond the ChatGPT knowledge cutoff date, which includes 254 code changes and 415 human comments. HYDRA-REVIEWER demonstrates great generalization ability on the unseen dataset. Additionally, we conduct a user study to further validate our experimental results. Finally, the cost analysis indicates that HYDRA-REVIEWER requires only an average of 0.018 dollars and 62.63 seconds to generate a review comment.

In general, this paper makes the following contributions:

- We manually read 1,920 comments written by real developers or generated by ACR methods, and summarize a taxonomy of code review dimensions and the limitations of existing ACR methods.
- We propose a new approach called HYDRA-REVIEWER, which leverages a collaborative multi-agent framework based on GPT-4o-mini to automatically generate review comments that are more comprehensive, correct, and specific. We extensively evaluate HYDRA-REVIEWER on the CodeReview and CodeReview-New datasets, and HYDRA-REVIEWER outperforms other ACR methods.
- We evaluate HYDRA-REVIEWER on an unseen dataset, and HYDRA-REVIEWER demonstrates great generalization ability.
- We conduct a user study to demonstrate the usability of HYDRA-REVIEWER.

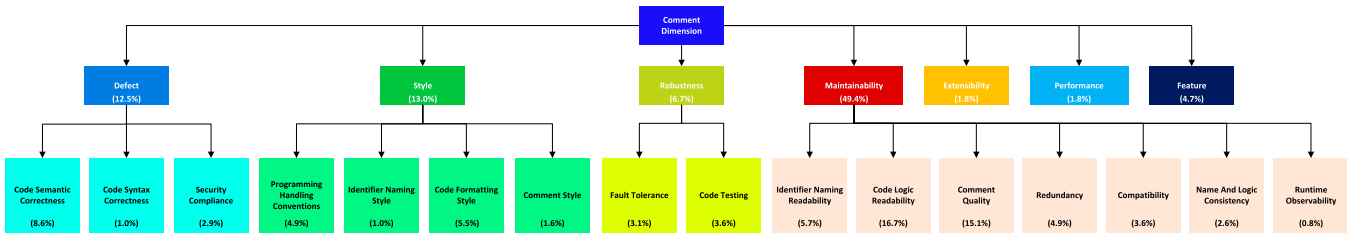

Fig. 2. Taxonomy of dimensions in code review.

- We publish our code, prompts and dataset for other researchers to replicate our study and conduct more future research[1].

II. Empirical Study

To investigate the dimensions of code review that real-world reviewers focus on, we manually analyze human comments and introduce a taxonomy of dimensions. Finally, we assess the severity of C1 (multi-dimensionality) and C2 (comment quality) in ACR methods.

A. Data Collection for Empirical Study

We select the top 1,000 projects on GitHub by stars. Then we collect 29,612 patches and corresponding 30,157 review comments from the pull requests of these projects using the GitHub REST API. The patch is a collection of all diffs for a file in a code change. In this paper, we treat a patch as a code change because it can more comprehensively represent the changes of a single file than a diff. Subsequently, we employ a statistical sampling method [13] to analyze MIN randomly selected comments and patches. In this empirical sutdy and subsequent experiments requiring sampling, MIN = 384, which ensures the estimated accuracy is within a 0.05 error margin at a 95% confidence level.

B. Dimensions Focused on in Code Review

To explore the dimensions that reviewers focus on in code reviews, the first two authors first independently analyze the 384 comments and create preliminary taxonomies. The team then discusses and merges two taxonomies into an initial taxonomy. Next, the first two authors independently label the dimensions covered by the comments. If none of the existing dimensions in the taxonomy can label a comment, the team discusses and proposes a new dimension, which is then added to the taxonomy. After completing the labeling, the first two authors compare their labeling results and discuss to resolve any discrepancies, determining a consistent labeling result. Ultimately, we establish a taxonomy containing 19 dimension categories, divided into seven groups (with the "Extensibility", "Performance", and "Feature" groups having no subcategories). The taxonomy is shown in Fig. 2. The remainder of this section will discuss the code review dimensions and illustrative examples.

[1]https://github.com/dcq01/Hydra-Reviewer

1) **Defect.** The comments in this dimension aim to identify and reveal potential defects in the code. We identify three subcategories.
   a) **Code Semantic Correctness (8.6%).** Comments in this subcategory identify cases where the code semantics is not correctly implemented as required. For example, "*I'm not sure if Ord can be implemented here properly. Are we potentially arbitrarily stating that all 'Window' variants are less than 'Image' variants?*"
   b) **Code Syntax Correctness (1.0%).** Comments in this subcategory identify syntax issues present in the code. For example, "*test functions have to start with test_ to get run. Please rename to test_delay.*"
   c) **Security Compliance (2.9%).** Comments in this subcategory indicate that the patches may introduce serious internal security vulnerabilities, such as memory leaks or data race, which may cause system failure or data loss. For example, "*This should be passed as a parameter to the function. These helpers can be called by multiple threads, which would induce a race condition writing to the variable here.*"
2) **Style.** The comments in this dimension indicate that the coding style of the changes is inconsistent with the project style. We have identified four subcategories.
   a) **Programming Handling Conventions (4.9%).** These review comments point out the failure to adhere to project-prescribed programming methods, processes, and conventions when implementing certain logic in the patches, with the purpose of ensuring that the modifications align with the project's best practices. For example, "*In general we don't use 'const' yet in the core JS libraries. Can you just use 'var' for now?*"
   b) **Identifier Naming Style (1.0%).** These review comments point out that the naming conventions for identifiers such as variables, functions, and classes in the patches do not conform to the project's naming style. For example, "*Variable and function names should follow the ['snake_case'] naming convention. Please update the following name accordingly: 'gArray'.*"
   c) **Code Formatting Style (5.5%).** These review comments point out that the use of spaces, indentation, line breaks, and parentheses in the patches does not

conform to the project's formatting style. For example, "*Trailing Whitespace Violation: Lines should not have trailing whitespace. (trailing_whitespace).*"

d) **Comment Style (1.6%).** These review comments point out that the placement of comments and the organization of the content within the comments do not conform to the project's comment style. For example, "*Please reformat this comment block to have a uniform width.*"

3) **Maintainability.** The comments in this dimension provide suggestions for improving code readability to enhance code maintainability. We identify seven subcategories.

a) **Identifier Naming Readability (5.7%).** Comments in this subcategory suggest that the readability of the names of certain variables, functions, classes, and other identifiers in the patches is insufficiently clear, and proposes possible renaming suggestions. For example, "*Please provide descriptive name for the parameter: 'x'.*"

b) **Code Logic Readability (16.7%).** Comments in this subcategory suggest that the code logic is not intuitive or easy to understand, and propose specific refactoring suggestions to simplify the code and improve its readability. For example, "*Instead of inlining lookup maps, I think it would be clearer to turn the whole thing into a switch.*"

c) **Comment Quality (15.1%).** Comments in this subcategory suggest adding or removing code comments, or making specific changes to existing comments, in order to improve their readability. For example, "*Could you add a comment for this 'sortBy'?*"

d) **Redundancy (4.9%).** Comments in this subcategory find that there are duplicate or unnecessary code blocks and logic in the code, and propose specific code refactoring suggestions to reduce code redundancy. For example, "*line 83-92 can be encapsulated into a function.*"

e) **Compatibility (3.6%).** Comments in this subcategory focus on the compatibility of the code across different versions, environments, or platforms, ensuring that patches do not introduce functional conflicts or disrupt existing system behavior, and support the coexistence of different components and versions. For example, "*It makes the interface incompatible to older versions.*"

f) **Name And Logic Consistency (2.6%).** Comments in this subcategory evaluate the correspondence and consistency between the code implementation and its contextual identifiers (such as function names, class names, and file names), and propose the necessary adjustments when inconsistencies are found. For example, "*I don't understand how the function being pure relates to whether it's needed for constant evaluation. I think this should move down to 'NeedDefinition' below?*"

g) **Runtime Observability (0.8%).** Comments in this subcategory suggest adding or modifying logging, assertions, or other mechanisms to help developers better observe and trace the system's behavior during execution. For example, "*It probably makes sense to log a message about that separately.*"

4) **Robustness.** The comments in this dimension focus on the stability of the code under various exceptional conditions and provide suggestions for testing and verification to enhance code robustness. We identify two subcategories.

a) **Fault Tolerance (3.1%).** These review comments focus on the code's ability to handle inputs that do not meet expectations and suggest measures such as validity checks and boundary validations to prevent the program from crashing or throwing exceptions due to non-malicious errors, thereby ensuring the system's stable operation. For example, "*You need to check 'nil', otherwise it could panic.*"

b) **Code Testing (3.6%).** These review comments require adding tests for newly implemented features or modifying existing tests to ensure that the tests cover the patches, thereby identifying potential issues in the code. For example, "*Can you add test cases when input file contains core-js imports ending with '.js'?*"

5) **Extensibility (1.8%).** The comments in this dimension assess whether the code has good extensibility to accommodate potential future requirement changes and propose suggestions for improving the code's design or structure, thus helping developers easily add new features or adjust the code when requirements change, which in turn improves development efficiency. For example, "*Personally I would prefer if all the interfaces on the GAL namespace kept accepting a 'ReadOnlySpan< byte>' as it makes the API more flexible, otherwise we *must* use a pooled buffer to pass the data, and passing data sourced from another place becomes difficult.*"

6) **Performance (1.8%).** The comments in this dimension assess the operational efficiency of the code, including but not limited to resource consumption, algorithmic efficiency, and response time. The comments point out the performance bottlenecks in the current code and provide suggestions for performance optimization. For example, "*Creating a buffer from 1MB of data takes about 700ms, wrapping it in a 'java.nio.ByteBuffer' takes only 3ms.*"

7) **Feature (4.7%).** The comments in this dimension propose new features to be implemented or suggest changes to existing features. For example, "*Perhaps we could hide this behind a 'show details' or something like that.*"

Furthermore, 15.1% of human comments do not cover any dimensions. These comments mainly include praise for the code and other content irrelevant to code quality, such as "Really?". In subsequent experiments, we will not label or count these comments with dimensions.

ChatGPT Prompt

As a code reviewer, imagine you are conducting a code review, and your team leader requests you to provide a review comment on a patch. The patch is a collection of diffs for a file within a code change. Please generate a review comment based on the patch. Your output should be a list of suggestions and refrain from adding meaningless words.

Generation Unit Prompt

As a code reviewer focused on the {Dimension}, you are required to meticulously review each line of the diff. After understanding the diff, evaluate whether there are {Issues in the Dimension}. If you believe there are {Issues in the Dimension}, output comments and specific suggestions. You must carefully check each line, with your attention focused solely on the {Dimension}. Please comment on the patch after fully understanding any additional information and the patch itself, and avoid commenting on the additional information. Only output comments related to {Dimension}. If the user provides critique, regenerate the comments based on the patch and the critique, and then only output the new comments.

Reflection Unit Prompt

You are a reviewer, and a user has commented on code changes based on the following requirements: {requirements}. The user's comment is your input. Please evaluate whether the user's comment meets the requirements. If you believe the user's comment meets the requirements, only output the identifier '<Good>'. If you think the user's comment does not meet the requirements or the content is too vague, provide your suggestions for improvement.

Clean-up Unit Prompt

You are a clean-up expert, and a user has commented on code changes based on the following requirements: {requirements}. The user's comment is your input. Please clean up any content in the comments that is irrelevant to the requirements, approval of the code changes, statements about the implementation of the code, or suggestions that lack practical significance. Retain any suggestions you find meaningful along with their corresponding code snippets. Only output the cleaned comment and refrain from adding meaningless words.

Fig. 3. Prompts of ChatGPT and different units.

C. Limitations of Existing ACR Methods

To validate the severity of C1 (multi-dimensionality) and C2 (comment quality) in ACR methods, we select three state-of-the-art code review comment generation methods: CodeReviewer [8], LLaMA-Reviewer [9], and GPT-4o-mini (denoted as ChatGPT). We instruct ChatGPT to act as a code reviewer and provide it with the definition of the patch, task details, and the output format. The prompt is shown in Fig. 3. We employ these three methods to generate comments for the 384 randomly selected patches. Additionally, to establish a benchmark reference for multi-dimensional evaluation, we also analyze human comments. The first two authors manually label the dimensions covered by ACR-generated comments and human comments. We calculate Cohen's Kappa [14] between the first two authors. Cohen's Kappa is a commonly used metric for measuring the agreement between two annotators in classification tasks. Its value ranges from -1 to 1, where a score between 0.6 and 0.8 indicates substantial agreement, and a score between 0.8 and 1.0 indicates almost perfect agreement [15]. The Cohen's kappa between the two authors is 0.95, indicating a very high level of agreement in their labeling results. For comments with inconsistent labels, the first two authors resolve discrepancies through discussion and relabel the comments to determine the final labels. For example, for the comment "*Please use camelCase for the function name.*", one author labeled it as related to "Identifier Naming Readability", while the other labeled it as "Identifier Naming Style". After discussion, both authors agreed that the comment focuses on whether the naming format complies with the project's naming conventions, rather than on the comprehensibility of the identifier name itself. Therefore, it was finally labeled as "Identifier Naming Style".

CodeReviewer and LLaMA-Reviewer generate comments at the diff level, while our study focuses on the patch level. Therefore, we standardize the granularity by splitting each patch into multiple diffs, using CodeReviewer and LLaMA-Reviewer to generate comments for each diff separately. Then, we merge these diff-level comments to form a comment for the entire patch. Additionally, we collect human comments from different locations within each patch. After collecting all the comments, the first two authors label the dimensions covered by ACR-generated comments and human comments. Table I presents the results, where the symbol "-" indicates missing values. "Average Number of Dimensions" refers to the average number of review dimensions covered by the comments generated by each method for a single patch. When counting the number of dimensions for a single patch, we sum the dimensions covered by all diff-level comments generated by CodeReviewer and LLaMA-Reviewer. For ChatGPT, we sum the dimensions covered by all suggestions in the generated suggestion list.

TABLE I
DIMENSIONS COVERED BY COMMENTS

Dimension	CodeReviewer	LLaMA-Reviewer	ChatGPT	Human Review
1a: Code Semantic Correctness	34.1%	39.1%	65.6%	23.2%
1b: Code Syntax Correctness	2.1%	1.0%	4.9%	2.3%
1c: Security Compliance	0.3%	-	14.1%	4.9%
2a: Programming Handling Conventions	2.9%	1.0%	26.0%	7.0%
2b: Identifier Naming Style	0.8%	-	21.1%	2.1%
2c: Code Formatting Style	9.9%	3.1%	20.8%	9.6%
2d: Comment Style	0.3%	-	49.0%	2.3%
3a: Identifier Naming Readability	13.8%	9.1%	28.6%	11.2%
3b: Code Logic Readability	12.8%	4.9%	48.2%	29.4%
3c: Comment Quality	7.3%	5.2%	93.0%	23.4%
3d: Redundancy	13.5%	4.9%	32.3%	11.5%
3e: Compatibility	1.6%	0.5%	28.9%	6.2%
3f: Name And Logic Consistency	3.1%	0.3%	13.0%	2.9%
3g: Runtime Observability	0.5%	1.0%	45.8%	1.6%
4a: Fault Tolerance	1.8%	-	58.6%	6.8%
4b: Code Testing	-	0.3%	59.4%	6.0%
5: Extensibility	0.3%	0.3%	38.0%	3.6%
6: Performance	0.3%	0.3%	25.0%	4.4%
7: Feature	0.5%	-	0.8%	8.1%
Average Number of Dimensions	1.1	0.7	6.3	1.7

Table I demonstrates that CodeReviewer and LlaMA-Reviewer generate comments covering fewer dimensions on average compared to human comments. While ChatGPT exhibits superiority in the number of dimensions covered, it fails to cover the dimensions focused on by human comments in 53.9% of the patches. Notably, ChatGPT provides improvement suggestions for dimensions not covered by human comments in 99.2% of the patches. This bidirectional discrepancy in dimension coverage indicates that both ACR-generated comments and human comments exhibit insufficient comprehensiveness.

Additionally, the first two authors identify ACR-generated comments deviating from developer expectations [12] through double-blind labeling (Cohen's Kappa = 0.80), and find that 65.6% of ACR-generated comments fail to meet developer expectations. To further analyze the characteristics of these comments, we follow the same process used to summarize the taxonomy of dimensions, identifying two primary limitations: "Incorrect" and "Vague". The "Vague" is further subdivided into three subcategories: "Lack of Comment Location", "Lack of Justification for Suggested Changes" and "Lack of Actionable Feedback".

1) **Incorrect:** The change suggestions proposed in the comments are incorrect. Implementing these suggestions would lead to syntax errors or alter the existing correct code logic. This issue is commonly found in comments

TABLE II
LIMITATIONS OF DIFFERENT METHODS

Limitation	CodeReviewer	LLaMA-Reviewer	ChatGPT
Incorrect	30.5%	35.9%	-
Lack of Comment Location	75.8%	81.5%	-
Lack of Justification for Suggested Changes	53.6%	71.9%	-
Lack of Actionable Feedback	-	-	16.4%

generated by CodeReviewer and LLaMA-Reviewer, indicating a lack of deep understanding of the code by these methods.

2) **Vague:** Comments are not detailed enough, reducing their effectiveness and increasing the comprehension cost for developers.
   a) **Lack of Comment Location:** Comments do not specify which line of the patch the feedback refers to. This makes the comments lose their practical value, as the developers do not know which part of the code the comments address. This issue is commonly found in comments generated by CodeReviewer and LLaMA-Reviewer.
   b) **Lack of Justification for Suggested Changes:** Comments provide suggested changes without offering reasons, making it difficult to convince the code author to adopt the suggested changes. This issue is commonly found in comments generated by CodeReviewer and LLaMA-Reviewer.
   c) **Lack of Actionable Feedback:** Comments suggest that a certain dimension of the code requires improvement but do not provide clear instructions or steps for modification, reducing the usefulness of comments. This issue is present in comments generated by ChatGPT.

The first two authors label the ACR-generated comments according to the definitions of the identified limitations. The results are presented in Table II.

It can be observed that comments generated by CodeReviewer and LLaMA-Reviewer generally lack comment locations and justifications for suggested changes, and 33.2% of the comments are incorrect. In the comments generated by ChatGPT, 16.4% lack actionable feedback. The above results indicate that a considerable portion of ACR-generated comments have quality issues, which makes it difficult to meet developers' expectations.

III. APPROACH

HYDRA-REVIEWER is a multi-agent review comment generation system. We first introduce an overview of HYDRA-REVIEWER, followed by a detailed explanation of each step.

A. Overview

The framework of HYDRA-REVIEWER is illustrated in Fig. 4. HYDRA-REVIEWER consists of three parts: context retrieval, initial comment generation, and comment processing. In the context retrieval, we design a component called context completer. The context completer extracts contextual information related to the patch from the source code by combining the LLM agent with an Abstract Syntax Tree parsing tool, which helps subsequent review agents refine their understanding of the patch. After obtaining the patch and contextual information, we generate initial comments. To address the challenge of insufficiently comprehensive comments generated by previous ACR methods, we extend our consideration to review patches from various dimensions. The recent developments in the field of LLM agents provide this opportunity, allowing different agents to review from different dimensions. In initial comment generation, each review agent examines the patch from its specified dimension. Agents employ reflection and a clean-up unit to ensure that the generated review comments do not deviate from the specified dimension. In comment processing, HYDRA-REVIEWER first summarizes multiple comments from review agents and outputs the summarized review comments in the form of a suggestion list, then filters out low-quality suggestions, reorders the suggestions according to dimension priorities, and ultimately outputs the processed comments. Comment processing helps developers avoid spending effort on low-quality suggestions and ensures that the most important issues are addressed first.

HYDRA-REVIEWER uses GPT-4o-mini in all modules involving LLMs.

B. Context Retrieval

In context retrieval, the context completer uses the Abstract Syntax Tree parsing tool tree-sitter [16] to collect context related to the patch from the source code.

As the length of the input context increases, the LLM exhibits performance degradation and tends to overlook critical information in long inputs [17]. Therefore, simply providing the entire source code of the patch to the LLM for code review may not fully leverage the powerful capabilities of the LLM. However, the patches lack many dependency details, and reviewing only the patch might make it difficult to identify some potential errors or generate incorrect review comments. Thus, we prompt the context completer to select the identifiers of code elements (i.e., functions, methods, and classes) that it may have doubts about in the patch, and require it to rank these identifiers in descending order according to its degree of lack of understanding of the code elements. This process fully relies on the LLM's own understanding capability. These identifiers are then fed into tree-sitter, which returns corresponding code snippets from the source code. We limit the selection to functions, methods, and classes because they can effectively provide contextual information while balancing the computational overhead of tree-sitter and the comprehension burden of the review agent. The retrieved code snippets are provided as additional contextual information alongside the patch to the subsequent review comment generation framework. These code snippets supplement the missing contextual information in the patch, enabling the LLM agent to understand the patch within a relatively more complete context, thereby reducing the risk of generating incorrect review comments.

C. Initial Comment Generation

To review the code in depth from various dimensions, we leverage the LLM's potential to simulate different agents. Each

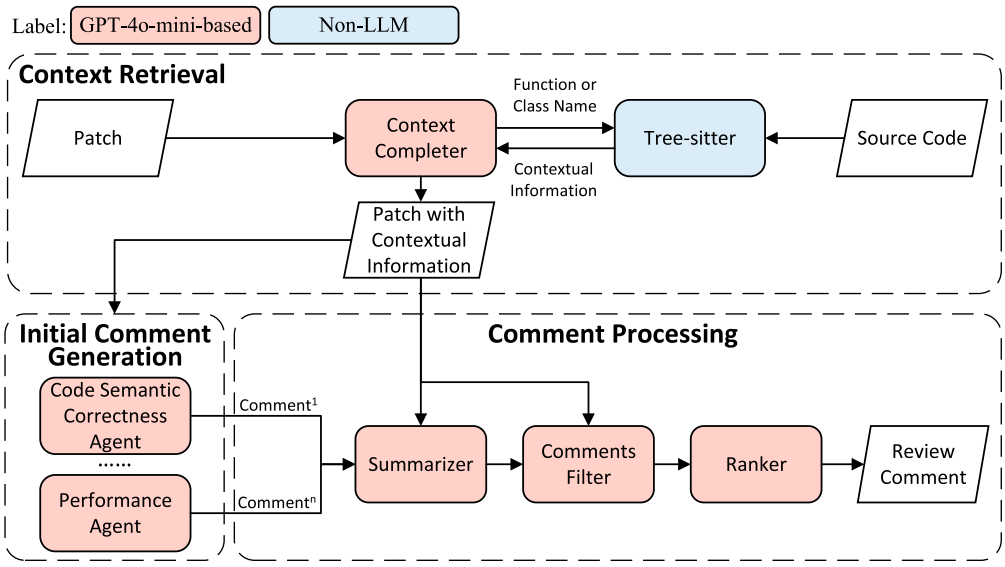

Fig. 4. Overall framework of HYDRA-REVIEWER.

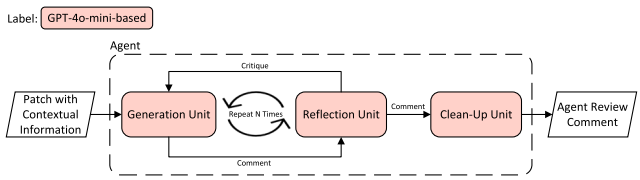

Fig. 5. The design of agent.

agent review the code from a unique dimension defined in the taxonomy. The design of the agent is illustrated in Fig. 5.

Since reviewing from the "Feature" dimension requires obtaining requirement information from the code repository, but it is currently challenging to accurately retrieve requirement information related to patches. Furthermore, the "Feature" dimension does not focus on the quality of the code. Given that our main focus is on reviewing the quality of the code, we do not instruct the agent to generate comments from the dimensions of "Feature". In subsequent experiments, we will no longer discuss or perform statistical analysis on "Feature" dimensions.

However, directly instructing the LLM agent to generate comments is difficult to achieve the expected results, as it may still focus on and provide comments from other dimensions even when required to generate comments from a specified dimension. Additionally, the generated comments may include unhelpful suggestions or natural language unrelated to the code review.

Therefore, we use reflection [18] to enable the LLM agent to better focus on its specified dimension. During the reflection process, there are two nodes: the generation unit and the reflection unit. Firstly, the generation unit will generate an initial comment according to the task requirements, and this initial comment is not directly output but instead passed to the reflection unit. The reflection unit evaluates the comment generated by the generation unit according to the task requirements. If the reflection unit deems that the comment does not satisfy the requirements, it will provide critiques. The generation unit then generates a new comment based on the received critiques. The reflection process repeats continuously until the reflection unit deems the comment to meet the requirements or it reaches the set maximum number of reflections. Through reflection, we can reduce the possibility of the agent does not review according to the specified dimensions. Furthermore, we implement a clean-up unit to clean the comment. The prompts for the three units are shown in Fig. 3.

Generation Unit. In the prompt for the generation unit, we specify a unique review dimension, elaborate on its definition and corresponding scenarios. We instruct the generation unit to review the code for any issues or shortcomings from the specified dimension, and explicitly require the generation unit to focus on its dimension to generate a corresponding comment. If critique is received, the generation unit will reflect on it and generate a new comment.

Reflection Unit. In the prompt for the reflection unit, we convey the tasks performed by the generation unit. Then, we instruct it to evaluate whether the generation unit has completed the task as required, based on the task requirements and the comment. The reflection unit also evalutes whether the comment is too vague. If the reflection unit determines that the generation unit did not review from the specified dimension or if the review comment is vague, it will generate a critique. Otherwise, it will generate "Good". The agent automatically decides, based on the content generated by the reflection unit, whether to enter a new round of reflection or pass the comment to the clean-up unit for cleaning.

Clean-up Unit. In the prompt for the clean-up unit, we similarly convey the tasks performed by the generation unit. We instruct it to remove suggestions in the comment that do not fall within the specified dimension, as well as unhelpful content such as praise for the patch and statements about code implementation. Finally, it outputs the cleaned comment, with the instruction not to add any natural language on its own.

The output of the clean-up unit serves as the final output of the agent.

D. Comment Processing

After obtaining 18 review comments generated by LLM agents, directly merging and outputting these comments does not produce ideal results. Since there are a large number of suggestions, developers need to spend considerable effort to understand the merged comment, which reduces efficiency. Additionally, there are quality issues within the comments. Therefore, the comments needs to be processed in order to obtain a high-quality review comment.

To address this, we design three components: a summarizer, a comment filter, and a ranker. The summarizer selects the specific suggestions it considers correct and capable of improving code quality from the comments of the 18 LLM agents, and summarizes them. The comment filter applies specified rules to filter the summarized comment, removing low-quality suggestions from the comment. Finally, to enable developers to prioritize their efforts on more important suggestions, the ranker reorders the suggestions according to dimension priorities. Due to space constraints, the full prompts for three components are provided in our open-source repository. Below, we will explain the design of each component in sequence.

Summarizer. Due to the hallucinations in LLM, the review comments generated by the agents may contain incorrect suggestions. Therefore, summarizing based solely on the review comments could result in an incorrect final review comment. To ensure the correctness of the final review comment, the input to the summarizer includes not only the review comments but also the patch and additional context information. We require the summarizer to verify the suggestions in the review comments based on the patch and summarize the suggestions it considers correct. Finally, the summarizer outputs the list of checked and summarized suggestions.

Comment Filter. Its input includes the summarized comment and the patch. Due to the large number of comments generated by the LLM agents and the fact that overly long inputs affect the LLM's task performance [17] (this is also verified in later experiments), it is difficult for the summarizer to meticulously check each comment, which may lead to quality issues still existing in the summarized comment. In contrast, the summarized comment is relatively concise, which greatly reduces the difficulty of verification. Therefore, we manually review 384 summarized comments and define five types of low-quality suggestions, then formulate the following filtering rules. Rules 1-3 are used to filter low-quality suggestions generated due to the inherent issues of the LLM. In response to the limitations of the ACR method identified in our empirical study, which include "Incorrect" and "Vague", we formulate rules 4 and 5.

1) **Unnecessary Suggestions:** We observe that the summarized comment contain numerous suggestions on dimensions such as "Comment Quality", "Identifier Naming Readability", and "Fault Tolerance". However, the code may have already been well implemented in these dimensions. For example, the code comments clearly describe the functionality of the corresponding code, but the agent still provides improvement suggestions on the "Comment Quality" dimension, which is included in the summarized comments. We refer to these suggestions as unnecessary suggestions. If a suggestion is unnecessary, it should be removed.
2) **Duplicate Suggestions:** The summarized comment may contain multiple suggestions that are expressed in different forms but propose the same improvement, i.e., duplicate suggestions. We require the comment filter to carefully evaluate the semantics of each suggestion and remove duplicate suggestions.
3) **Descriptive Suggestions:** If a suggestion merely restates the implementation of the patch, it should be removed.
4) **Incorrect Suggestions:** If a suggestion is an incorrect suggestion proposed by the agent based on a misunderstanding of the patch, it should be removed.
5) **Vague Suggestions:** The suggestion only indicates the need to improve the quality of a particular dimension of the code, but does not provide justifications or actionable feedback. If a suggestion is vague, it should be removed.

LLMs can correct their own output errors through a reflection framework [18], [19], which indicates that the LLM-based comment filter has the ability to effectively check the comments generated by the summarizer. Therefore, we prompt the comment filter to filter the suggestions in the summarized comment according to the filtering rules. This process fully relies on the LLM's own understanding capability.

Ranker. To enable developers to prioritize more important suggestions, we instruct the ranker to reorder the suggestions in the list based on dimension priorities. We refer to the usefulness rankings for different categories of review comments provided by Turzo et al. [20]. Since our taxonomy differs from their classification perspective, the first two authors independently mapped our dimensions to their categories of comments based on Turzo et al.'s descriptions, resolved mapping disagreements through discussion (Cohen's kappa = 0.90), and ultimately established dimension priorities. Table III shows the dimension priorities, the comment categories from Turzo et al., and the descriptions of these categories. The ranker reorders the suggestion list based on the dimensional priorities and outputs the reordered suggestion list as the final output of HYDRA-REVIEWER.

IV. EXPERIMENTAL SETUP

This section provides a detailed description of our experimental setup, outlining the datasets, the baselines, the evaluation metrics, the research questions, and our implementation details.

A. Data Collection

We utilize the CodeReview (CR) [8] and CodeReview-New (CRN) [3] datasets. CodeReview is a widely used dataset for code review tasks, comprising 9 programming languages

TABLE III
PRIORITY OF DIMENSIONS. A HIGHER PRIORITY LEVEL NUMBER INDICATES HIGHER PRIORITY

Priority Level	Dimension	Category	Description
P10	Fault Tolerance	Validation	Validation mistakes or mistakes made when detecting an invalid value are of this class. Any kind of user data sanitization-related comments are in this category, too.
P9	Code Semantic Correctness	Logical	Control flow, comparison related, and logical errors.
P8	Compatibility	Interface	Mistakes when interacting with other parts of the software such as existing code library, hardware device, database or operating system.
P7	Performance	Solution Approach	Suggestions to adopt an alternate algorithm or data structure.
P6	Security Compliance	Resource	Resource (variables, memory, files, database) initialization, manipulation, and release.
P5	Comment Quality	Documentation	Suggestions to add/modify comments or documentation to aid code comprehension.
P4	Runtime Observability	Alternate Output	Comments that suggest modifying the error message, toast message, alert, or change what is returned by a function.
P3	Identifier Naming Style	Naming Convention	Violations of identifier naming conventions.
P2	Code Formatting Style	Visual Representation	Whitespace, blank lines, code rearrangements, and indentation-related comments.
P1	Others	Other	Comments not belonging to any of the above categories.

and 829 repositories. We use only the test set from CodeReview. Considering that CodeReview contains low-quality comments [3], CodeReview-New builds on CodeReview and collects newer data from the same repositories (with data later than CodeReview but all earlier than October 2023) and additionally adds 472 repositories of seven new programming languages (such as Swift, Objective-C, and Kotlin). CodeReview and CodeReview-New have no overlapping samples, and the two datasets collectively contain 28,049 samples.

In CodeReview and CodeReview-New datasets, the review comments for samples are single comments targeting only one location within the diffs. However, our research focuses on generating review comments for single patches. Therefore, we use the GitHub REST API to trace all comments for the same patch and crawl the modified source code after the commits. A small number of samples encountered "URL Not Found" issues during the tracing process, which leads to the inability to crawl the relevant data. Consequently, we have to remove these samples. Ultimately, we construct an initial dataset containing 24,910 patches and 38,674 comments.

Since Tree-sitter currently does not support the four programming languages, Swift, SQL, Perl, and PHP, we have removed samples of these languages from the dataset. In Section II-B, we note that 15.1% of comments are irrelevant to code quality and require no evaluation. Furthermore, in Section III-C, we state that code review from the "Feature" dimension is unnecessary. By labeling and removing these two categories of comments, the final dataset contains 18,138 patches and 25,323 comments.

B. Comparison Baselines

We choose three methods used in our empirical study: CodeReviewer, LLaMA-Reviewer and ChatGPT. In addition, we include Comprehensive-ChatGPT (C-ChatGPT) and DeepSeek-V3 as supplementary baselines for comparison.

CodeReviewer: It adopts the T5 architecture, with a total of 223 million parameters. The model is initialized using the weights of CodeT5 and is pretrained on three tasks: Diff Tag Prediction, Denoising Objective, and Review Comment Generation. We use the open-source CodeReviewer model.

LLaMA-Reviewer: It uses the smallest version of LLaMA (only 6.7B parameters) and employs both prefix-tuning [21] and LoRA tuning [22] for parameter-efficient fine-tuning (PEFT) on the automated code review task. In this research, we use the LLaMA model fine-tuned with LoRA and set the rank to 16, which represents the best-performing setting reported in the original paper.

ChatGPT: We use the latest GPT-4o-mini model via the OpenAI API [23], which is a streamlined version of the GPT-4o model released by OpenAI, offering a good balance of performance and cost-effectiveness.

Comprehensive-ChatGPT (C-ChatGPT): For the **C1** and **C2** challenges that appear in the empirical study of ChatGPT, it may be due to the simplicity of the prompt design. Therefore, we combine the prompts used in HYDRA-REVIEWER to generate comments for each dimension into a single comprehensive prompt that covers all dimensions, to verify whether providing detailed dimension definitions enables ChatGPT to overcome these two challenges.

DeepSeek-V3: It adopts a Mixture-of-Experts (MoE) architecture and has 671B parameters. It performs well on reasoning and programming tasks, with performance comparable to mainstream closed-source LLMs such as GPT-4o [24].

C. Evaluation Metrics

BLEU. It is a commonly used metric for evaluating the quality of text generated by translation models [25]. We use the BLEU-4 variant, which calculates the final BLEU-4 score by counting the number of 4-grams that match between the generated comment and the ground truth comment, and applying smoothing and penalty mechanisms. The score ranges from 0% to 100%. A higher BLEU-4 score indicates a higher degree of consistency between the generated comment and the ground truth comment.

However, there are limitations to using only the BLEU score. For example, a higher BLEU score may result from a method targeting the low-hanging fruits [26]. Additionally, the BLEU score relies on exact word matching rather than understanding the sentence semantics. Since the comment generated by ChatGPT and HYDRA-REVIEWER may contain additional code or explanations, its longer generated length may result in a lower BLEU score. This leads to the situation where, even though the generated comment is semantically close to the ground truth comment, BLEU may underestimate its quality, thus making the measurement inaccurate. Therefore, it is insufficient for BLEU score to fully evaluate the true helpfulness and readability of the comments. To address this, we also conduct a qualitative evaluation of the ACR-generated comments based on the research questions and further explore the helpfulness and readability of the ACR-generated comments in the subsequent user study.

Kendall's Tau. It is a statistic used to measure the consistency of ranking between two lists [27]. Kendall's Tau calculates the ordering relationships of all data pairs between the two lists and computes the differences to determine the consistency between the lists, with values ranging from -1 to 1. A value of 1 indicates that the order of the two lists is completely consistent, while -1 indicates that the order is completely opposite. By calculating Kendall's Tau between the actual order of suggestions and the expected order in a comment, we can quantify the degree of consistency between the suggestions and the priority order. In our ablation study, we use Kendall's Tau-b variant to quantify the consistency between the comments before and after reordering and the priority order, and compare these two values to assess the effectiveness of the ranker. The Kendall's Tau-b is specifically designed to handle ties in the data (i.e. repeated values).

D. Research Questions

In this paper, we investigate the following four research questions to evaluate the performance of HYDRA-REVIEWER:

RQ1: How capable is HYDRA-REVIEWER in generating review comments in quantitative evaluation? We use HYDRA-REVIEWER and baselines to generate review comments on the dataset and perform quantitative evaluation of the review comments to compare the performance of different methods.

RQ2: Are the comments generated by HYDRA-REVIEWER more comprehensive? We manually evaluate and compare the number of dimensions in human comments, comments generated by ChatGPT, C-ChatGPT, DeepSeek-V3, and HYDRA-REVIEWER, while also assessing whether the comments generated by these four ACR methods contain suggestions that are more important than human comments. In this research question, we do not include CodeReviewer and LLaMA-Reviewer, as our empirical study has already shown that the comments generated by these two methods are not comprehensive.

RQ3: What are the contributions of each component in HYDRA-REVIEWER? We compare the capability of HYDRA-REVIEWER to generate review comments before and after removing different components, to investigate whether each component affects the quality of generated comments. We construct four variants of HYDRA-REVIEWER: first, the removal of context completer (denoted as w/o-CC); second, the elimination of the comment filter (denoted as w/o-CF); third, the exclusion of the ranker (denoted as w/o-R); and fourth, the removal of agent's internal processing (denoted as w/o-AIP), where the agent generates comments directly from the generate unit without reflection and clean-up. We conduct quantitative and qualitative analyses on the comments generated by these four variants to investigate the specific impact of each component.

RQ4: How is the generalization capability of HYDRA-REVIEWER? The patches and review comments in the datasets may have been used by GPT-4o-mini. To explore the performance of HYDRA-REVIEWER on unseen dataset, we collect data from January 1, 2024, onwards, to construct a new dataset, as the training data for GPT-4o-mini extends only up to October 2023 [28]. We use HYDRA-REVIEWER and baselines to generate comments on the unseen dataset and evaluate the generalization capability of HYDRA-REVIEWER through quantitative and qualitative evaluation.

E. Implementation Details

We implement HYDRA-REVIEWER in Python based on the most popular multi-agent framework LangChain [29], and use the GPT-4o-mini model from the ChatGPT family as the engine for HYDRA-REVIEWER. The OpenAI API is accessed in May 2025, with a total cost of 326 dollars. The cost analysis of different methods will be further elaborated in Section V-E. In most cases, lower temperature settings tend to produce better and more stable results [3], so we set the temperature to 0. Furthermore, research shows that the performance growth gradually diminishes when the maximum number of reflections exceeds 2 [30]. Considering the computational cost and time overhead, we set the maximum number of reflections to 2 as the default experimental setting. The experiments for CodeReviewer and LLaMA-Reviewer are conducted on an NVIDIA A100-SXM4-80GB GPU platform.

V. EXPERIMENTAL RESULTS

In this section, we sequentially address each research question based on the experimental results. Our discussion begins with an evaluation of HYDRA-REVIEWER's capability to generate review comments, followed by an analysis of the implications arising from various components. We also analyze the generalization capability of HYDRA-REVIEWER on the unseen dataset. Finally, we report the cost analysis of different methods.

A. RQ1 Quantitative Evaluation of Hydra-Reviewer

For different ACR methods, we adopt different BLEU score calculation approaches. CodeReviewer and LLaMA-Reviewer generate comments sequentially for single diffs within a patch. Therefore, we first identify which diff in the patch corresponds

TABLE IV
QUANTITATIVE EVALUATION RESULTS

Dataset	Samples	Method	BLEU
CR+CRN	18,138	CodeReviewer	5.02
		LLaMA-Reviewer	4.96
		ChatGPT	6.87
		C-ChatGPT	6.52
		DeepSeek-V3	7.85
		HYDRA-REVIEWER	**8.20**
Ablation Dataset	384	w/o-CC	9.09
		w/o-CF	**9.82**
		w/o-R	9.32
		w/o-AIP	8.29
		HYDRA-REVIEWER	9.33
Unseen Dataset	254	CodeReviewer	3.63
		LLaMA-Reviewer	3.89
		ChatGPT	5.86
		C-ChatGPT	5.54
		DeepSeek-V3	6.58
		HYDRA-REVIEWER	**6.82**

to the ground truth comment, then extract the comments generated by these two methods for that diff and compute the BLEU score between ACR-generated comments and the ground truth comment. ChatGPT, C-ChatGPT, HYDRA-REVIEWER, and DeepSeek-V3 generate comments as a list of suggestions, so we split the comments by suggestion and compute the BLEU score for each suggestion with the ground truth comment, taking the maximum score as the final score for that ground truth comment.

For an ACR method, after obtaining the BLEU scores between each ground truth comment and the ACR-generated comments, we calculate the average score of the ground truth comments corresponding to each patch to serve as the score for that patch. Finally, we take the average score of all patches to represent the overall performance of the method on the dataset.

Table IV shows that HYDRA-REVIEWER surpasses all baselines on the datasets. Among them, ChatGPT demonstrates a significant improvement over CodeReviewer and LLaMA-Reviewer, increasing by 36.9% and 38.5% respectively, indicating that ChatGPT already possesses strong code understanding and reasoning capabilities. However, the BLEU score of C-ChatGPT is 5.1% lower than that of ChatGPT, which may be due to the fact that the prompt includes definitions for all dimensions, making it excessively lengthy and thereby degrading the performance of the LLM. In contrast, the BLEU score of HYDRA-REVIEWER is 19.4% higher than that of ChatGPT, demonstrating the effectiveness of the multi-agent framework. Moreover, even Deepseek-V3 with its 671B parameters achieves a BLEU score that is 4.3% lower than HYDRA-REVIEWER. These results indicate that conducting reviews from more dimensions enables better generation of review comments whose semantics are closer to high-quality ground truth comments.

B. RQ2 Comprehensiveness of Review Comments

To evaluate whether ACR-generated comments are comprehensive, we compare the number of dimensions covered by the human comments, ChatGPT, C-ChatGPT, DeepSeek-V3, and HYDRA-REVIEWER-generated comments on the same patch. The first two authors manually label 544 human comments, 2,315 suggestions from ChatGPT-generated comments, 2,199 suggestions from C-ChatGPT-generated comments, 6,214 suggestions from Deepseek-V3-generated comments, and 4,473 suggestions from HYDRA-REVIEWER-generated comments corresponding to 384 randomly selected patches. The Cohen's kappa between the two authors is 0.95.

The results, as shown in Table V, indicate that the dimensions covered by comments generated by different methods for the same patch are significantly more than those covered by human comments. Furthermore, HYDRA-REVIEWER's comments cover the largest number of dimensions on average, with 1.3 more dimensions than those of ChatGPT. Table V also presents the distribution of dimensions covered by comments generated using different methods. Since we found in our empirical study that the ACR methods may generate low-quality suggestions (which have no practical value because they may be incorrect or vague), it is necessary to remove these low-quality suggestions and re-calculate the number of dimensions covered by the comments to ensure the statistical authenticity. To control the cost of manual labeling, the first two authors randomly sample 384 suggestions from the review comments of each method on the CR and CRN datasets, and label each suggestion as either low-quality or not. The labeling achieves a Cohen's kappa of 0.72, which indicates a high level of agreement between the two authors. In Table V, we report the proportion of low-quality suggestions for each dimension in parentheses. The results show that HYDRA-REVIEWER generates the lowest proportion of low-quality suggestions. In addition, HYDRA-REVIEWER can provide suggestions from more dimensions, such as "Security Compliance", "Fault Tolerance", and "Performance", which ChatGPT tends to focus less on. However, this does not guarantee that users will trust HYDRA-REVIEWER, and we will further investigate this in our user study.

Additionally, we focus on whether ACR methods could provide suggestions that are more important than the human comments. We compare the suggestions in the comments generated by different methods with the human comments, based on the priorities of dimensions defined in ranker. The results are shown in Table V. For each patch, the comments generated by ChatGPT contain an average of 2.5 suggestions with higher-priority than the human comments. In contrast, HYDRA-REVIEWER performs more impressively, with an average of 5.8 higher-priority suggestions in its comments. Although DeepSeek-V3 generates a slightly greater number of higher-priority suggestions than HYDRA-REVIEWER, considering its large parameter size and higher proportion of low-quality suggestions, HYDRA-REVIEWER still demonstrates strong overall performance.

The results show that through the multi-agent framework, HYDRA-REVIEWER can effectively review from more dimensions. Furthermore, HYDRA-REVIEWER can identify more important potential optimizations or defects than human reviewers and ChatGPT, which also demonstrates the importance of multi-dimensional reviews.

TABLE V
DISTRIBUTION OF COMMENT DIMENSIONS

Dataset	CR+CRN						Unseen Dataset				
Dimension	**Human Review**	**ChatGPT**	**C-ChatGPT**	**DeepSeek-V3**	**HYDRA-REVIEWER**	**w/o-AIP**	**Human Review**	**ChatGPT**	**C-ChatGPT**	**DeepSeek-V3**	**HYDRA-REVIEWER**
1a: Code Semantic Correctness	26.0%	68.8% (34.7%)	54.2% (54.3%)	69.8% (24.6%)	66.9% (10.0%)	63.8%	35.8%	77.2%	57.1%	63.0%	75.2%
1b: Code Syntax Correctness	1.3%	4.2% (33.3%)	5.5% (20.0%)	5.7% (25.0%)	7.6% (0.0%)	5.5%	2.4%	5.9%	5.9%	6.3%	10.6%
1c: Security Compliance	1.0%	13.5% (36.4%)	9.6% (75.0%)	27.3% (21.6%)	51.8% (0.0%)	34.4%	3.1%	19.3%	9.4%	18.9%	50.8%
2a: Programming Handling Conventions	7.8%	31.8% (27.6%)	17.4% (43.8%)	33.6% (20.5%)	44.3% (4.5%)	35.7%	4.7%	39.4%	16.1%	31.5%	53.9%
2b: Identifier Naming Style	1.6%	20.1% (50.0%)	13.0% (55.6%)	18.2% (36.4%)	12.2% (25.0%)	5.7%	1.6%	23.2%	15.4%	8.3%	16.1%
2c: Code Formatting Style	5.2%	15.4% (40.0%)	26.6% (62.5%)	22.4% (0.0%)	11.5% (40.0%)	8.1%	8.7%	17.7%	22.0%	14.6%	13.8%
2d: Comment Style	1.0%	55.5% (22.5%)	40.4% (54.8%)	51.0% (21.6%)	30.7% (17.6%)	23.4%	1.2%	50.8%	37.8%	47.2%	25.2%
3a: Identifier Naming Readability	6.8%	32.0% (20.7%)	48.4% (56.5%)	32.6% (30.3%)	25.5% (29.4%)	16.7%	6.7%	35.8%	54.7%	19.3%	28.0%
3b: Code Logic Readability	19.0%	39.3% (25.0%)	56.0% (16.1%)	54.4% (5.2%)	57.3% (0.0%)	62.8%	24.0%	47.6%	57.1%	53.9%	63.0%
3c: Comment Quality	13.5%	88.3% (18.2%)	88.3% (25.0%)	71.9% (14.3%)	37.8% (11.1%)	33.9%	10.6%	88.2%	89.8%	74.4%	40.2%
3d: Redundancy	6.8%	24.5% (15.8%)	29.7% (25.0%)	39.8% (0.0%)	34.9% (0.0%)	41.4%	11.0%	29.1%	40.6%	32.3%	39.8%
3e: Compatibility	0.8%	29.9% (45.8%)	31.0% (67.9%)	38.5% (26.0%)	31.8% (23.1%)	25.5%	4.3%	38.6%	32.7%	34.6%	36.6%
3f: Name And Logic Consistency	2.1%	20.3% (20.0%)	21.9% (56.0%)	28.1% (32.0%)	31.5% (11.8%)	27.9%	2.0%	26.4%	21.7%	28.7%	38.2%
3g: Runtime Observability	3.9%	48.7% (21.4%)	44.5% (32.4%)	49.7% (7.1%)	85.4% (1.4%)	84.6%	3.1%	48.4%	47.6%	46.5%	80.3%
4a: Fault Tolerance	7.0%	57.0% (11.7%)	63.3% (20.6%)	62.8% (7.5%)	90.6% (0.8%)	93.0%	5.5%	56.7%	66.5%	60.2%	93.7%
4b: Code Testing	2.3%	56.2% (53.7%)	41.7% (75.0%)	61.5% (29.9%)	72.9% (10.0%)	83.6%	5.9%	56.3%	40.6%	40.9%	75.6%
5: Extensibility	3.9%	29.7% (23.3%)	28.6% (6.7%)	42.7% (10.2%)	47.4% (0.0%)	58.3%	4.3%	33.9%	36.6%	43.3%	55.1%
6: Performance	2.1%	18.2% (40.0%)	13.0% (16.7%)	43.8% (10.5%)	40.6% (3.4%)	48.4%	5.5%	21.3%	16.5%	27.2%	39.4%
7: Feature	-	-	-	-	-	-	-	-	-	-	-
Average Number of Dimensions	1.1	6.5	6.3	7.5	**7.8**	7.5	1.4	7.2	6.7	6.5	**8.4**
Average Number of Higher-Priority Suggestions	-	2.5	2.4	**6.2**	5.8	4.9	-	2.5	2.5	2.4	**6.0**
Proportion of Low-Quality Suggestions	-	29.7%	37.2%	17.7%	**8.6%**	-	-	-	-	-	-

C. RQ3 Component Effectiveness

To investigate the effectiveness of different components, we use the 384 patches selected in RQ2 as an ablation dataset and evaluate the quality of comments generated by different variants on the ablation dataset.

1) Context Completer: To explore whether context completer enhances HYDRA-REVIEWER's understanding of patches, we conduct a preliminary comparison of 100 comments generated by both w/o-CC and HYDRA-REVIEWER, and we summarize the impact of this component on the comments as two advantages:

1) Reducing the number of redundant suggestions output by LLM agents.
2) Enabling LLM agents to identify more potential optimizations or defects.

Redundant suggestions refer to suggestions for changes that have already been implemented in the source code outside the patch. For example, in Fig. 6(a), w/o-CC discovers through the diff header that the added code is within the "pearsonr" function, and infers that the code author intends to calculate the Pearson correlation coefficient. Since the Pearson correlation coefficient requires the input data (x and y) to have the same length, w/o-CC suggests adding a check to ensure that the lengths of x and y are equal. However, in the source code, there is already a check to ensure that the lengths of x and y are equal, but this check does not appear in the patch. Therefore, w/o-CC, which only review the patch, proposes this redundant suggestion. If the additional context information containing the check had been provided along with the patch for review by the LLM agents, it is possible to avoid generating redundant suggestions.

Fig. 6(b) illustrates an example of HYDRA-REVIEWER discovering a potential optimization point based on the additional context information. Through the additional context information, HYDRA-REVIEWER finds that the logic for handling the negative "idx" in the added code is also implemented in other functions within the source code. Therefore, it suggests encapsulating this logic into a method to reduce code redundancy and improve code maintainability.

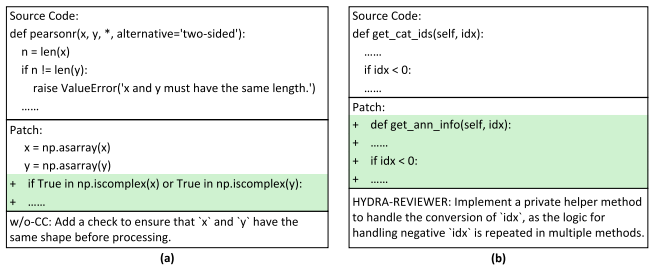

Fig. 6. Examples of context completer impact on suggestions.

TABLE VI
IMPACT OF CONTEXTUAL RETRIEVAL

Method	Redundant Suggestions	Potential Optimization Points or Defects
w/o-CC	42	-
HYDRA-REVIEWER	**8**	63

To evaluate these two advantages, we split the suggestions in the comments generated by w/o-CC and HYDRA-REVIEWER on the dataset, and randomly select 384 suggestions from each, for a total of 768 suggestions. The first two authors manually label and count the number of redundant suggestions present in these 768 suggestions, as well as the number of suggestions where HYDRA-REVIEWER discovers potential optimization points or defects using additional context information. The Cohen's kappa is 0.88. Table VI shows that compared to w/o-CC, HYDRA-REVIEWER reduces the output of redundant suggestions by 34, representing a decrease of 81.0%, and proposes 63 suggestions that required additional context information to identify potential optimization points or defects, accounting for 16.4% of the comments. The results indicate that context completer effectively avoids HYDRA-REVIEWER outputting redundant suggestions while assisting it in discovering potential optimization points or defects.

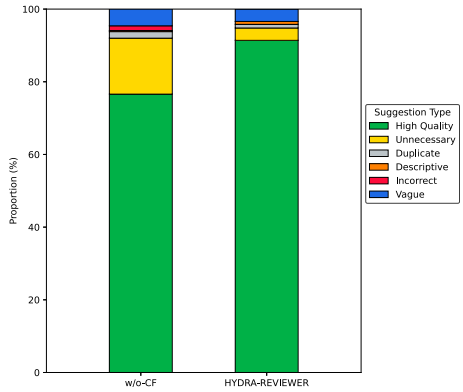

Fig. 7. Distribution of suggestion types.

Additionally, we calculate the BLEU scores of the comments generated by different variants and compare them with HYDRA-REVIEWER. Table IV shows that, compared to HYDRA-REVIEWER, the BLEU score of w/o-CC decreases by 0.24.

The quantitative and qualitative evaluation indicates that it is insufficient to review only the patch, as some issues require context related to the patch to be discovered. By providing additional context information through context completer, HYDRA-REVIEWER can better understand the patch and generate more high-quality review comments.

2) Comment Filter: To investigate whether the comment filter reduces the proportion of low-quality suggestions in the comments, we split the comments generated by w/o-CF and HYDRA-REVIEWER, and randomly select 384 suggestions from each, for a total of 768 suggestions. Based on the filtering rules in the comment filter, the first two authors manually label 768 suggestions to determine whether they are low-quality and identify their specific types. The Cohen's kappa is 0.84.

Fig. 7 shows the quantity and proportion of different types of suggestions in the 768 suggestions. In the suggestions generated by HYDRA-REVIEWER, low-quality suggestions account for 8.6%, which is lower than the proportion of low-quality suggestions generated by w/o-CF, which is 23.4%. The comment filter effectively filters out unnecessary suggestions, while for other types of low-quality suggestions—due to their inherently lower proportion—the filtering effectiveness is less noticeable. However, overall, the proportion of all types of low-quality suggestions shows a reduction. In addition, the comments generated by HYDRA-REVIEWER contain an average of 11.7 suggestions, while the comments from w/o-CF contain an average of 16.2 suggestions. The results show that, although the BLEU score of HYDRA-REVIEWER is slightly lower than that of w/o-CF, the comment filter effectively filters out low-quality suggestions in the comments, preventing developers from expending effort on understanding low-quality suggestions. This also indirectly suggests that the BLEU score cannot fully evaluate the quality of the review comments.

3) Ranker: To investigate whether the ranker can rank suggestions in the comments by priority, we calculate the Kendall's Tau between the order of suggestions and the expected order for each comment generated by w/o-R and HYDRA-REVIEWER. The average Kendall's Tau of comments generated by HYDRA-REVIEWER is 0.48, which is 0.11 higher than that of w/o-R. A paired t-test on the Kendall's Tau values across all patches ($n = 384$) indicates that this difference is statistically significant ($t = 4.737$, $p = 3.06 \times 10^{-6}$). Specifically, HYDRA-REVIEWER achieves higher Kendall's Tau for 229 patches. The results indicate that the ranker can effectively prioritize the suggestions in the comments. The comments generated by HYDRA-REVIEWER align better with the expected order, enabling developers to prioritize their efforts on addressing high-priority suggestions. Additionally, the BLEU score of w/o-R differs from that of HYDRA-REVIEWER. We observe that due to the instability of LLM, w/o-R may change the expression of the suggestions when ranking, but the semantics of the suggestions remain unchanged.

4) Agent's Internal Processing: To investigate whether the agent's internal processing helps HYDRA-REVIEWER achieve multi-dimensional review, the first two authors manually label the dimension of each suggestion in the comments generated by w/o-AIP. The Cohen's kappa is 0.94.

As shown in Table V, the comments generated by w/o-AIP cover an average of 7.5 dimensions, which is close to HYDRA-REVIEWER. However, HYDRA-REVIEWER proposes a higher number of higher-priority suggestions. Furthermore, as seen in Table IV, HYDRA-REVIEWER's BLEU score is higher than that of w/o-AIP, improving 1.04.

The results indicate that the multi-agent framework already enables HYDRA-REVIEWER to review from more dimensions effectively, making the advantage of the reflection and clean-up in increasing the number of dimensions covered by the comments less apparent. However, the improvement in BLEU score indicates that the reflection and clean-up enable the LLM agent to conduct a deeper review on the patch from the specified dimension and generate a higher-quality review comment.

D. RQ4 Generalization Capability of Hydra-Reviewer

We employ a stratified sampling method to collect data after January 1, 2024, to construct an unseen dataset. First, we select 191 active repositories from the CodeReview and CodeReview-New datasets (those that have released at least 10 pull requests from January 1, 2024, to the present). Then, for each repository, we randomly select 2 pull requests. For each pull request, we randomly select 1 human comment and crawl the relevant data. After processing these data in the same way as the benchmark dataset, we ultimately obtain a dataset containing 254 patches and 415 human comments.

Table IV shows the performance of different methods on the unseen dataset. Compared to the benchmark dataset used in this study, all ACR methods' BLEU scores decrease. To investigate whether this result is due to the increased complexity of the data in the unseen dataset, we calculate the average character length of patches and human comments in the unseen dataset and compare these values with those in the benchmark dataset. The statistical results show that the average character lengths of

patches and human comments in the unseen dataset are 3,532 and 179, respectively, which are 924 and 44 higher than those in the benchmark dataset. This indicates that the patches and human comments in the unseen dataset are more complex, making it more challenging for ACR methods to generate comments semantically consistent with the human comments, which leads to a decline in BLEU scores for all six methods.

Even though the BLEU scores have decreased, the performance gaps among the six methods remain consistent with the results in RQ1, with HYDRA-REVIEWER still outperforming the other methods. This indicates that HYDRA-REVIEWER's higher BLEU score on the benchmark dataset compared to baselines is not due to data leakage.

Similar to RQ2, the first two authors label the dimensions covered by the comments generated by different methods on the unseen dataset and count the number of covered dimensions and the number of higher-priority suggestions in the comments. The Cohen's kappa between the two authors is 0.94. The results are shown in Table V. Overall, the results are consistent with those in RQ2. Both the average number of dimensions covered by the HYDRA-REVIEWER-generated comments and the average number of higher-priority suggestions proposed by HYDRA-REVIEWER are the highest among all ACR methods. In contrast, DeepSeek-V3's average number of higher-priority suggestions has significantly decreased, which may be due to data leakage.

Based on the results of the quantitative and qualitative evaluation, we find that HYDRA-REVIEWER performs well on the unseen dataset and outperforms all baselines, demonstrating strong generalization ability.

E. Cost Analysis

Table VII shows the average time and cost required by different ACR methods to review a patch. Same as in RQ2, we do not include CodeReviewer and LLaMA-Reviewer in the cost analysis. The results show that Hydra-Reviewer has the highest average time and cost. To further identify potential efficiency bottlenecks in Hydra-Reviewer, we measure the time consumption of different components inside Hydra-Reviewer. The results show that the multi-agent framework and reflection process account for the majority of the time cost. In the future, we can try to reduce the time cost by optimizing prompts or reducing redundant inferences of AI units. It is worth emphasizing that although Hydra-Reviewer has higher costs, Hydra-Reviewer based on GPT-4o-mini still generates review comments with higher quality than the stronger model DeepSeek-V3, demonstrating the effectiveness of the multi-agent framework in improving model code review performance

TABLE VII
COST ANALYSIS OF DIFFERENT METHODS

Method	Component	Time (s)	Cost (USD)
ChatGPT	—	4.63	0.0002
C-ChatGPT	—	6.39	0.0004
DeepSeek-V3	—	18.83	0.0060
HYDRA-REVIEWER	Context Completer	3.51	0.0002
	Review Agents	35.77	0.0153
	Summarizer	7.62	0.0013
	Comment Filter	8.69	0.0006
	Ranker	7.04	0.0004
	Total	**62.63**	**0.0178**

VI. USER STUDY

We conduct a user study to evaluate the usefulness of HYDRA-REVIEWER in assisting developers with efficient code review.

A. User Study Design

We select patches from Python, Java, and C++ samples in the dataset. These three languages are the most popular programming languages, and participants are likely to be more familiar with them, thereby avoiding a decrease in the quality of code review due to unfamiliarity with the programming language. To investigate whether users can effectively review code from multiple dimensions with the help of different ACR methods, we select 17 patches, and the human comments corresponding to these patches cover 18 dimensions. We provide participants with our taxonomy of dimensions and require them to review each patch from different dimensions and give review comments in the form of a list of suggestions. Participants can freely access any online resources, including search engines and programmer forums, to help understand the patches, but can not use ChatGPT or other online LLM for understanding or directly reviewing patches. In addition, participants are required to record the time consumption reviewing each patch.

We recruit 21 master's students to participate in the user study. They are equally divided into 7 groups (i.e., G-1, G-2, G-3, G-4, G-5, G-6, G-7), with 3 participants in each group. G-1 is required to independently review the patches, while we provide G-2, G-3, G-4, G-5, G-6, and G-7 with comments generated by CodeReviewer, LLaMA-Reviewer, ChatGPT, C-ChatGPT, DeepSeek-V3, and HYDRA-REVIEWER for reference.

We evaluate the performance of different groups of participants from three aspects: the number of dimensions covered in the review comments, which reflects whether the ACR method helps participants review from more dimensions; the number of patches for which they provide suggestions semantically consistent with at least one ground truth comment, which measures the group's ability to effectively identify issues across different dimensions; and the time taken to complete the review, which evaluates whether the ACR method can effectively help participants reduce the time spent on code review.

Additionally, each participant is required to score the comments generated by the 6 ACR methods for these 17 patches from the perspectives of helpfulness and readability, with a score range from 1 to 5, where 1 indicates very poor and 5 indicates very good.

B. Result

The first two authors independently judge whether each suggestion in the participants' review comments is relevant to code quality or is correct, and resolve any disagreements through discussion. If a suggestion is relevant to code quality and is correct, they further evaluate the dimensions it covers.

TABLE VIII
RESULTS OF USER STUDY

Group	Number of Dimensions	Patch Count	Time Consumption (second)	Helpfulness						Readability					
				CodeReviewer	LLaMA-Reviewer	ChatGPT	C-ChatGPT	DeepSeek-V3	HYDRA-REVIEWER	CodeReviewer	LLaMA-Reviewer	ChatGPT	C-ChatGPT	DeepSeek-V3	HYDRA-REVIEWER
G-1	1.18	1.33	360.90	2.16	2.12	3.82	3.35	4.16	4.33	2.51	2.35	3.78	3.43	4.20	4.24
G-2	1.20	2.67	359.45	2.31	2.00	3.67	3.20	3.76	3.86	3.47	3.41	4.12	4.16	4.25	4.12
G-3	0.76	3.00	**315.10**	1.69	1.45	3.78	3.73	4.22	4.71	2.98	2.71	4.43	4.29	4.45	4.76
G-4	3.06	6.00	438.92	1.96	1.88	4.02	3.65	4.57	4.51	3.12	3.25	4.12	3.63	4.80	4.24
G-5	2.02	5.33	861.02	1.71	1.63	4.04	4.12	4.45	4.35	1.78	1.51	3.88	4.20	4.25	4.18
G-6	2.10	5.33	531.35	2.57	2.35	3.67	3.59	4.12	4.35	2.78	2.88	3.47	3.73	4.29	4.18
G-7	**3.73**	**10.67**	497.06	1.92	1.88	3.63	3.31	4.31	4.33	2.16	2.14	3.98	3.41	3.98	4.27
Average	2.01	4.90	443.82	2.05	1.91	3.82	3.57	4.24	**4.36**	2.69	2.62	3.98	3.85	**4.33**	4.30

The Cohen's kappa is 0.90. Table VIII shows the results of the user study.

1) Average Number of Dimensions Covered in Review Comments: The "Number of Dimensions" column in Table VIII represents the average number of dimensions covered in the review comments provided by different groups. In G-1, which does not use ACR methods, the comments on each patch cover an average of only 1.18 dimensions. The numbers of dimensions for G-2 and G-3 are similar to that of G-1, further indicating that CodeReviewer and LLaMA-Reviewer have limitations in providing comprehensive review comments. In contrast, G-4, G-5, and G-6 have dimension counts of 3.06, 2.02, and 2.10, respectively, which are higher than that of G-1 but still lower than G-7, which uses HYDRA-REVIEWER. These results indicate that, compared to other ACR methods, HYDRA-REVIEWER is more effective in identifying issues across more dimensions.

2) Number of Patches with Suggestions Consistent with Real Review Comments: The "Patch Count" in Table VIII represents the average number of patches where participants in different groups provide suggestions semantically consistent with at least one ground truth comment. The "Patch Count" of G-2 and G-3 are 2.67 and 3.00, respectively, both higher than that of G-1, indicating that although CodeReviewer and LLaMA-Reviewer struggle to provide comprehensive review comments, the comments they generate still have some reference value. G-4, G-5, and G-6 have "Patch Count" of 6.00, 5.33, and 5.33, respectively, indicating that ChatGPT and DeepSeek-V3 already have a good ability to review code from multiple dimensions. Notably, the "Patch Count" of G-7 is significantly higher than that for the other groups, at 10.67. This result shows that by applying the context completer and multi-agent framework, HYDRA-REVIEWER is able to conduct deeper reviews across various dimensions, discovering more important issues rather than merely providing superficial suggestions.

3) Time Consumption: The "Time Consumption" in Table VIII refers to the time taken by different groups to complete the code review task. G-3 completes the task the fastest, averaging 315.10 seconds. In contrast, G-4, G-5, G-6, and G-7 show longer time consumptions-438.92, 861.01, 531.35, and 497.06 seconds respectively—all higher than the first three groups. We attribute this difference to the fact that the comments generated by ChatGPT, C-ChatGPT, DeepSeek-V3, and HYDRA-REVIEWER contain more suggestions (average suggestion count: 7.20), requiring participants to spend additional time understanding these comments. Although the time required to complete the task is longer, HYDRA-REVIEWER helps participants propose more in-depth suggestions across multiple dimensions, making the additional time investment worthwhile.

4) User Scores: The "Helpfulness" and "Readability" in Table VIII represent the helpfulness and readability scores given by different groups for each ACR method. The helpfulness scores of CodeReviewer and LLaMA-Reviewer are similar, averaging 1.98, indicating that participants consider the helpfulness of these two methods to be poor. ChatGPT has a helpfulness score of 3.82, indicating that directly using ChatGPT for code review already demonstrates promising results. In contrast, HYDRA-REVIEWER has the highest helpfulness score of 4.36, which indicates that participants consider HYDRA-REVIEWER to be the most helpful.

In terms of readability, the scores of CodeReviewer and LLaMA-Reviewer are both below 3, indicating poor readability. Benefiting from the advanced natural language processing capabilities of LLMs, the comments generated by ChatGPT, C-ChatGPT, DeepSeek-V3, and HYDRA-REVIEWER demonstrated considerably better readability. Although HYDRA-REVIEWER's readability score is slightly lower than that of DeepSeek-V3, it still outperforms direct use of ChatGPT, showing a notable improvement.

In conclusion, HYDRA-REVIEWER can effectively help reviewers conduct in-depth code review across more dimensions. Participants suggest that HYDRA-REVIEWER outperforms the baselines in both helpfulness and readability. Notably, although HYDRA-REVIEWER is based on GPT-4o-mini, it still demonstrates superior performance compared to the stronger model (DeepSeek-V3). Moreover, despite HYDRA-REVIEWER relying on an LLM that may generate hallucinated content, the user study results still show that users exhibit greater trust in it.

C. Feedback From GitHub Developers

To explore the applicability of HYDRA-REVIEWER in real development workflows, we select the top 100 GitHub repositories ranked by star count. In each repository, we randomly choose one pull request and use the REST API to fetch the patch and source code of the last commit of that pull request. Then, we generate review comments using HYDRA-REVIEWER. We send the review comments to the corresponding developers and ask two brief Yes/No questions to facilitate quick feedback:

1) Does this comment provide suggestions from a dimension you hadn't considered?
2) Do you find this comment helpful?

We receive a total of 30 responses. Among them, 14 developers give clear answers to both questions (hereafter referred to as valid responses). Among the valid responses, 8

developers believe the comments raise dimensions they had not considered, and 6 developers find the comments helpful. We further summarize the reasons why some responses do not answer "Yes" to both questions. The main reasons include: some developers hold reservations about AI-generated comments; some HYDRA-REVIEWER-generated comments are general and lack specific details; some commits involve minor changes and developers think complex suggestions involving multiple dimensions are unnecessary; and some review comments go beyond the scope of the pull request.

In summary, although some comments need improvement in specificity or scope, overall, HYDRA-REVIEWER can still provide helpful review comments from dimensions developers have not considered, showing its practical value in real development workflows.

VII. RELATED WORK

In this section, we summarize related works from two aspects: automated code review and LLM-based agents.

A. Automated Code Review

Previous studies show that code review is an important component of the software development process, but developers need to invest a significant amount of time in this activity [4], [31]. Regarding automated code review (ACR) activities, researchers have conducted extensive studies, such as comment location prediction [5], [6] and review comment recommendation [32], [33], emphasizing their importance and potential impact [34].

Li et al. divided the ACR pipeline into three tasks [8]: (1) Review Necessity Prediction: The ACR model predicts whether a diff requires a review comment, thus filtering out code that does not need a review comment. (2) Review Comment Generation: After the code author submits code for review, the model provides possible review comments for the reviewer as a reference. (3) Code Refinement: After the reviewer provides review comments, the model modifies the code based on the code and the review comments, and outputs the modified code. In this paper, we focus on the review comment generation task because it is the most critical step in the code review activities.

Early studies on code review comments (CRC) primarily employed retrieval-based methods to extract historical comments. Gupta et al. recommended comments based on the relationship between new code snippets and code changes [32]. Siow et al. enhanced retrieval by capturing semantic information through an attention-based LSTM [33]. With the rise of deep learning, the field shifted toward generating CRC. Tufano et al. pretrained a Text-To-Text Transfer Transformer model on code and technical language to generate CRC [7]. Subsequently, CodeReviewer [8] and AUGER [35] further improved the performance by using a pre-trained model specifically for code review and review tags. LLaMA-Reviewer utilized the LLaMA model and parameter-efficient fine-tuning to generate CRC, achieving performance comparable to state-of-the-art code review methods with fewer resources [9].

Previous studies focus only on single comments for diffs, resulting in ACR methods typically generating comments that cover only a few dimensions of code review. In practice, human reviewers need to review the code change across multiple dimensions [8]. Our study can complement the research on automated CRC generation. At the same time, we combine LLM that have strong code understanding capabilities with a multi-agent framework to conduct a comprehensive review of the code from various dimensions.

B. LLM-Based Agents

LLM-based agents aim to autonomously solve complex tasks specified in natural language, such as website tasks [36], social behavior simulation [37], and interactive writing [38]. AutoGPT [39] and BabyAGI [40] can conduct arbitrary missions based on user instructions. ReAct [41] and Reflection [18] utilize a chain of thought prompts to generate reasoning trajectories and action plans with LLMs. Both works demonstrate the effectiveness of the ReAct-style reasoning loop as a design paradigm in enhancing agent reasoning capabilities. Additionally, AgentFL [42] and AutoCodeRover [43] allow agents to use external tools to retrieve specific information from the project, enabling them to better leverage context and thereby enhance their abilities, demonstrating efficiency in complex tasks such as fault localization and automated program improvement, respectively.

Multi-agent frameworks based on LLMs have gained widespread recognition. Many works enhanced the problem-solving capabilities of LLMs by introducing interactions between multiple agents [44], [45], [46]. Stable-Alignment created instruction datasets by obtaining consensus on value judgments through interactions with LLM agents in a sandbox [47]. Li et al. instructed agents to re-query from multiple semantic perspectives to enhance the adaptability and robustness of search engine ranking models to different query [48]. Recently, multi-agent systems such as MetaGPT [19] and ChatDev [49] have begun to shift towards automated software development.

We instruct the LLM agents to use external tools to extract context related to the patch from the source code, in order to enhance the LLM agents' understanding of the patch. Through the multi-agent framework, HYDRA-REVIEWER can review the code from multiple dimensions. We design different components to address challenges faced in multi-agent collaboration, thereby improving the quality of the final review comments.

VIII. THREATS TO VALIDITY

A. Internal Validity

Our approach is based on ChatGPT, which means that the randomness in ChatGPT's predictions may threaten the validity of the results. To address this, we evaluate using a large number of samples to ensure the reliability of the results. Additionally, the design of the prompts also influences the results, and we try to follow existing best practices to address this issue. The subjectivity of human judgment during the manual labeling process is another potential threat to the validity of our results.

To address this, the first two authors independently label each sample and resolve any inconsistencies through discussion. The first author, with more than 10 years of programming experience and 5 years of software development experience in IT companies, and the second author, with over 5 years of programming experience, contribute their expertise to ensure the reliability of the labeling process.

B. External Validity

Our findings may not generalize beyond the dataset used in this study. Since the dataset is collected from Github, it is built upon only open-source projects. Therefore, our findings may not fully apply to industrial projects or other contexts. In future work, we plan to extend the datasets to include proprietary or industrial projects to further validate the generalizability of our findings. In addition, since Tree-sitter currently does not support Swift, SQL, Perl, and PHP, the conclusions do not cover these languages. Future evaluations can be added once Tree-sitter extends its language support. It is also worth noting that future code review dimensions may go beyond the current coverage of HYDRA-REVIEWER. For these new dimensions, we can summarize their definitions and design corresponding review agents, which can be integrated into the multi-agent framework of HYDRA-REVIEWER to support the continuous expansion of review dimensions.

C. Construct Validity

We use the BLEU-4 score to evaluate the quality of the generated comments. Although it is widely used in prior research [8], [9], it is not universally considered a suitable metric for evaluating code review comments because it only considers lexical matching of the text and ignores the sentence semantics. Therefore, we also perform qualitative evaluation of ACR-generated comments. In addition, we conduct a user study to evaluate the helpfulness and readability of the ACR-generated comments.

IX. CONCLUSION

In this paper, we first conduct an empirical study to assess the challenges faced by ACR methods. We propose a taxonomy for code review dimensions and identify the limitations of ACR-generated comments. To address these limitations, we introduce HYDRA-REVIEWER, a GPT-4o-mini-based multi-agent framework for generating CRC. HYDRA-REVIEWER generates multi-dimensional CRC through three steps: context retrieval, initial comment generation, and comment processing. Experimental results show that HYDRA-REVIEWER outperforms other ACR methods and effectively mitigates the limitations of existing ACR methods. Our user study further validates the potential of HYDRA-REVIEWER in assisting reviewers with code review. Finally, the cost analysis shows that HYDRA-REVIEWER requires only an average of 0.018 dollars and 62.63 seconds to generate a review comment.

REFERENCES

[1] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2013, pp. 712–721.

[2] M. Fagan, "A history of software inspections," in *Proc. Softw. Pioneers: Contributions Softw. Eng.*, 2002, pp. 562–573.

[3] Q. Guo et al., "Exploring the potential of ChatGPT in automated code refinement: An empirical study," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[4] A. Bosu and J. C. Carver, "Impact of peer code review on peer impression formation: A survey," in *Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas.*, Piscataway, NJ, USA: IEEE Press, 2013, pp. 133–142.

[5] V. J. Hellendoorn, J. Tsay, M. Mukherjee, and M. Hirzel, "Towards automating code review at scale," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2021, pp. 1479–1482.

[6] S.-T. Shi, M. Li, D. Lo, F. Thung, and X. Huo, "Automatic code review by learning the revision of source code," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, 2019, pp. 4910–4917.

[7] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2291–2302.

[8] Z. Li et al., "Automating code review activities by large-scale pre-training," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2022, pp. 1035–1047.

[9] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, "LLaMA-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 647–658.

[10] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.

[11] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[12] Z. Li, J. Chen, Q. Mao, X. Hu, K. Liu, and X. Xia, "Studying practitioners' expectations on clear code review comments," 2024, *arXiv:2410.06515*.

[13] R. Singh and N. S. Mangat, *Elements of Survey Sampling*, vol. 15, Dordrecht, Netherlands: Springer Science & Business Media, 2013.

[14] M. L. McHugh, "Interrater reliability: The kappa statistic," *Biochemia Medica*, vol. 22, no. 3, pp. 276–282, 2012.

[15] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, Mar. 1977.

[16] "Tree-sitter," 2024. Accessed: May 30, 2024. [Online]. Available: https://tree-sitter.github.io/tree-sitter/

[17] N. F. Liu et al., "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Ling.*, vol. 12, no. 1, pp. 157–173, 2024.

[18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023. Accessed: May 16, 2024. [Online]. Available: https://arxiv.org/abs/2303.11366

[19] S. Hong et al., "MetaGPT: Meta programming for multi-agent collaborative framework," 2023, *arXiv:2308.00352*.

[20] A. K. Turzo and A. Bosu, "What makes a code review useful to OpenDev developers? An empirical investigation," *Empirical Softw. Eng.*, vol. 29, no. 6, pp. 1–6, 2024.

[21] R. Zhang et al., "LLaMA-adapter: Efficient fine-tuning of language models with zero-init attention," 2023, *arXiv:2303.16199*.

[22] E. J. Hu et al., "Lora: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.

[23] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 27730–27744.

[24] D. Guo et al., "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.

[25] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Ling.*, 2002, pp. 311–318.

[26] R. Tufano, O. Dabić, A. Mastropaolo, M. Ciniselli, and G. Bavota, "Code review automation: Strengths and weaknesses of the state of the art," *IEEE Trans. Softw. Eng.*, vol. 50, no. 2, pp. 338–353, Feb. 2024.

[27] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, nos. 1–2, pp. 81–93, 1938.

[28] OpenAI, 2023. Accessed: Jul. 25, 2024. [Online]. Available: https://platform.openai.com/docs/models

[29] H. Chase, 2022. Accessed: Mar. 21, 2024. [Online]. Available: https://github.com/hwchase17/langchain

[30] J. Sun et al., "Think-on-graph: Deep and responsible reasoning of large language model with knowledge graph," 2023, *arXiv:2307.07697*.
[31] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: A case study at google," in *Proc. 40th Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2018, pp. 181–190.
[32] A. Gupta and N. Sundaresan, "Intelligent code reviews using deep learning," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD) Deep Learn.,* 2018.
[33] J. K. Siow, C. Gao, L. Fan, S. Chen, and Y. Liu, "Core: Automating review recommendation for code changes," in *Proc. IEEE 27th Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 284–295.
[34] C. Watson, N. Cooper, D. N. Palacio, K. Moran, and D. Poshyvanyk, "A systematic literature review on the use of deep learning in software engineering research," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, pp. 1–58, 2022.
[35] L. Li et al., "Auger: automatically generating review comments with pre-training models," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2022, pp. 1009–1021.
[36] I. Gur et al., "A real-world webagent with planning, long context understanding, and program synthesis," 2023, *arXiv:2307.12856*.
[37] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol.*, 2023, pp. 1–22.
[38] W. Zhou et al., "RecurrentGPT: Interactive generation of (Arbitrarily) long text," 2023, *arXiv:2305.13304*.
[39] "AutoGPT," 2024. [Online]. Available: https://github.com/Significant-Gravitas/AutoGPT
[40] "BabyAIG," 2024. [Online]. Available: https://github.com/yoheinakajima/babyagi
[41] S. Yao et al., "React: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.
[42] Y. Qin et al., "AgentFL: Scaling LLM-based fault localization to project-level context," 2024, *arXiv:2403.16362*.
[43] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutocoDerover: Autonomous program improvement," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2024, pp. 1592–1604.
[44] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," 2023, *arXiv:2305.14325*.
[45] M. Zhuge et al., "Mindstorms in natural language-based societies of mind," 2023, *arXiv:2305.17066*.
[46] E. Akata, L. Schulz, J. Coda-Forno, S. J. Oh, M. Bethge, and E. Schulz, "Playing repeated games with large language models," 2023, *arXiv:2305.16867*.
[47] R. Liu et al., "Training socially aligned language models in simulated human society," 2023, *arXiv:2305.16960*.
[48] X. Li et al., "Agent4ranking: Semantic robust ranking via personalized query rewriting using multi-agent LLM," 2023, *arXiv:2312.15450*.
[49] C. Qian et al., "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Ling. (Volume 1: Long Papers)*, 2024, pp. 15174–15186.